Logics and Symbolic AI
Planning

Nils Holzenberger

October 2, 2026

Outline

1. The monkey, the box and the banana
   - Planning problem
   - The STRIPS representation
   - STRIPS solver
2. Planning as satisfiability
   - Blocks world
   - Sussman anomaly
   - Kautz and Selman, 1992
   - From STRIPS to SAT
3. Partially Observable Markov Decision Process

Outline

1. The monkey, the box and the banana
   - Planning problem
   - The STRIPS representation
   - STRIPS solver
2. Planning as satisfiability
   - Blocks world
   - Sussman anomaly
   - Kautz and Selman, 1992
   - From STRIPS to SAT
3. Partially Observable Markov Decision Process

The task

- Description: see `https://ailab.r2.enst.fr/LKR/TP2.html`
- Famous toy problem in planning
- The world can be fully described by its *state*
- It is possible to go from one state to another using *actions*
- This vocabulary is identical in *reinforcement learning*
- *Planning* is the task of finding a sequence of actions going from initial to final state

Stanford Research Institute Problem Solver

- Stanford Research Institute Problem Solver (STRIPS), 1971
- Planning strategy, and also formal representation for problems
- The basic principles of this language are still in use

Knowledge representation

Describe the state of the world

- Monkey is in `a`, box is in `b`, banana is at `c`
  `At(a)`, `BoxAt(b)`, `BananaAt(c)`, `Level(low)`
- Monkey is in `a`, box is in `a`, monkey is on top of box, banana is at `c`
  `At(a)`, `BoxAt(a)`, `BananaAt(c)`, `Level(high)`
- Monkey is in `b`, box is in `a`, monkey holds banana
  `At(b)`, `BoxAt(a)`, `Have(banana)`
- Predicates are capitalized words, constants are lowercase words

Knowledge representation

Initial state: monkey is at a, on the floor, box is at c, banana is at b.
`BananaAt(b), At(a), Level(low), BoxAt(c)`

Goal state: monkey is at a and has the banana.
`Have(banana), At(a)`

Knowledge representation

Monkey can move from `X` to `Y`

```
Move(X, Y)
Preconditions:  At(X), Level(low)
Postconditions: !At(X), At(Y)
```

Predicates are capitalized words, constants are lowercase words, variables are capitalized words.

Knowledge representation

Climbing up the box

```
ClimbUp(Location)

Preconditions: At(Location), BoxAt(Location), Level(low)

Postconditions: Level(high), !Level(low)
```

Knowledge representation

Climbing down from the box

```
ClimbDown(Location)

Preconditions: At(Location), BoxAt(Location), Level(high)

Postconditions: Level(low), !Level(high)
```

Knowledge representation

Moving the box around

```
MoveBox(X, Y)

Preconditions: At(X), BoxAt(X), Level(low)

Postconditions: BoxAt(Y), !BoxAt(X), At(Y), !At(X)
```

Knowledge representation

Taking the banana

```
TakeBanana(Location)

Preconditions: BananaAt(Location), At(Location), Level(high)

Postconditions: Have(banana)
```

Finding the solution

Run the program

Run the trace

Backtracking:

- it can solve any problem
- it can spend time exploring pointless strategies
- it's up to programmer to encode knowledge about useless strategies to avoid exploring them

Planning with STRIPS

- Encode relevant knowledge
- Run the solver long enough to find a solution or realize there is none (semi-decidability)
- To speed things up:
  - write better program,
  - exploit corner cases, or
  - build better hardware

Outline

1. The monkey, the box and the banana
   - Planning problem
   - The STRIPS representation
   - STRIPS solver
2. Planning as satisfiability
   - Blocks world
   - Sussman anomaly
   - Kautz and Selman, 1992
   - From STRIPS to SAT
3. Partially Observable Markov Decision Process

Blocks

Blocks world

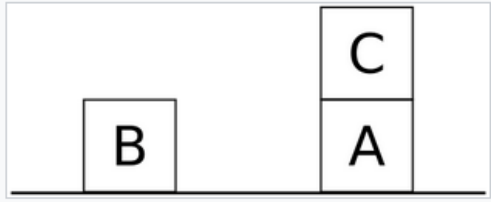
3 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

This article is about the general concept in computer science research. For the sandbox video game, see Blocksworld.

The **blocks world** is a planning domain in artificial intelligence. It consists of a set of wooden blocks of various shapes and colors sitting on a table. The goal is to build one or more vertical stacks of blocks. Only one block may be moved at a time: it may either be placed on the table or placed atop another block. Because of this, any blocks that are, at a given time, under another block cannot be moved. Moreover, some kinds of blocks cannot have other blocks stacked on top of them.[1]



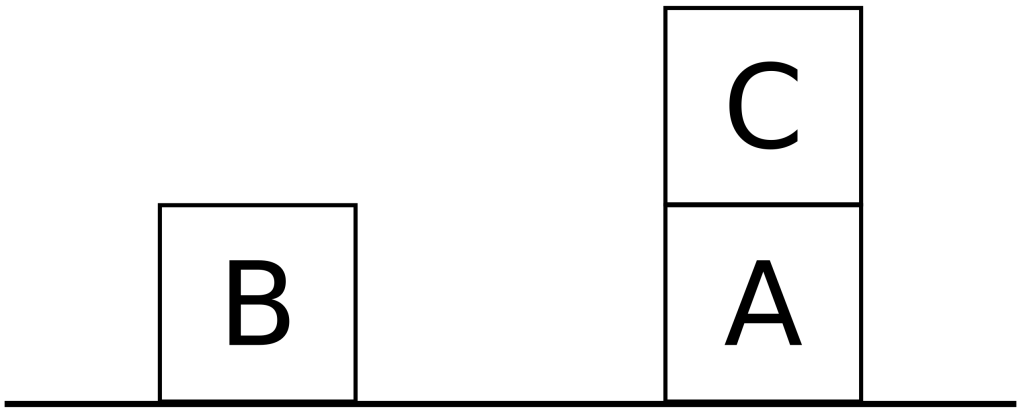
Step 1 of the Sussman anomaly, a problem in which an agent must recognise the blocks and arrange them into a stack with A at the top and C at the bottom

The simplicity of this toy world lends itself readily to classical symbolic artificial intelligence approaches, in which the world is modeled as a set of abstract symbols which may be reasoned about.

Blocks

Blocks can be stacked on top of one another

There is a special block called `t` (for *table*)



`Table(t)`, `On(b,t)`, `Clear(b)`, `On(a,t)`, `On(c,a)`, `Clear(c)`

Blocks actions

Move block from atop one block to another block

```
MoveBlocks(BlockMoved, BlockFrom, BlockTo)
Pre: !Table(BlockMoved), Clear(BlockMoved), Clear(BlockTo),
     On(BlockMoved, BlockFrom)
Post: On(BlockMoved, BlockTo), !On(BlockMoved, BlockFrom),
      !Clear(BlockTo), Clear(BlockFrom)
```

Blocks actions

Move block from table to another block

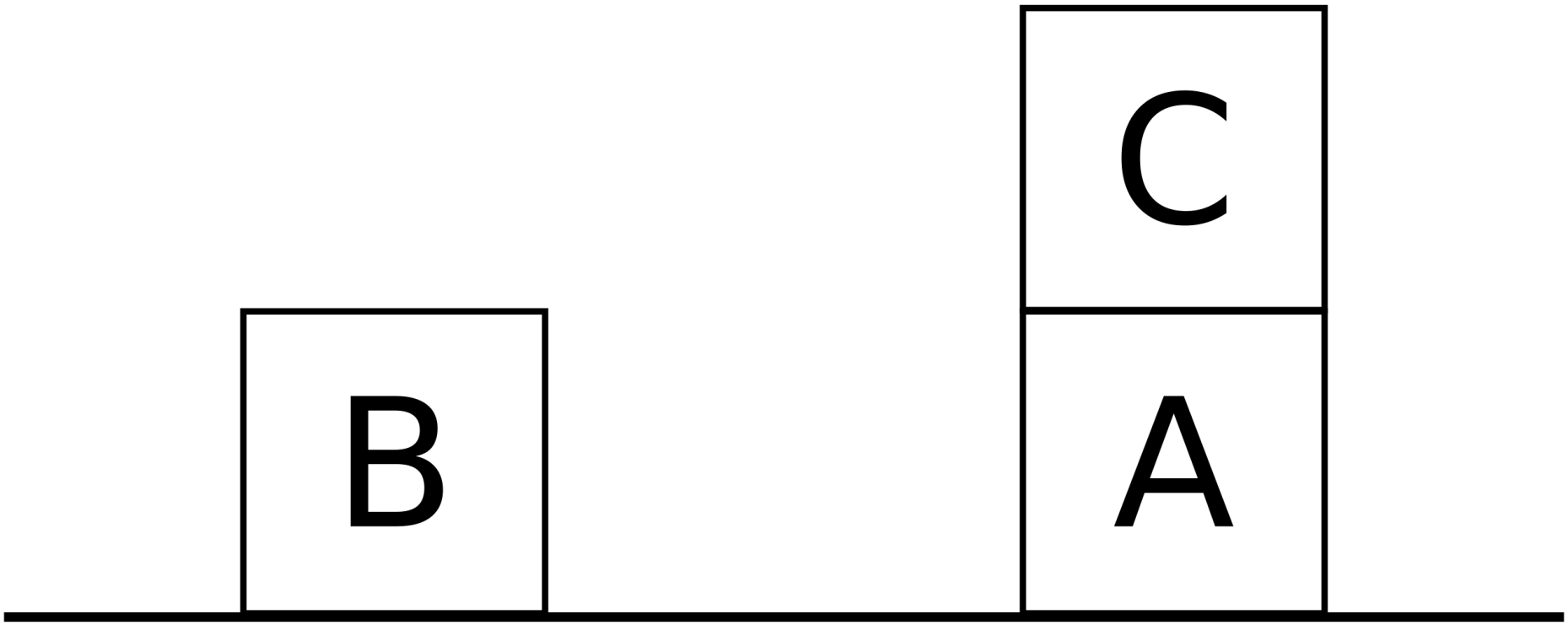
```
MoveFromTable(BlockMoved, BlockFrom, BlockTo)
Pre: !Table(BlockMoved), Table(BlockFrom), Clear(BlockMoved),
     Clear(BlockTo), On(BlockMoved, BlockFrom)
Post: On(BlockMoved, BlockTo), !On(BlockMoved, BlockFrom),
      !Clear(BlockTo)
```

Blocks actions

Move block from block to table

```
MoveToTable(BlockMoved, BlockFrom, BlockTo)
Pre: !Table(BlockMoved), Table(BlockTo), Clear(BlockMoved),
     On(BlockMoved, BlockFrom)
Post: On(BlockMoved, BlockTo), !On(BlockMoved, BlockFrom),
      Clear(BlockFrom)
```

The Sussman anomaly



The Sussman anomaly

Run `sussman.txt`

- Initial state:
  `Table(t), On(b,t), Clear(b), On(a,t), On(c,a), Clear(a)`
- Goal state: `On(a,b), On(b,c)`
- Move `b` onto `c`. Then `a` is not clear. So clear `a`. But then `b` is under `c`...
- What if we invert the two goals? Same problem

Planning

- Context: yearly competitions on planning
- Henry Kautz and Bart Selman, *Planning as satisfiability*, European conference on Artificial intelligence 1992
- Express a *planning* problem as a *SAT* problem, such that a valid plan corresponds to a solution to the SAT problem
- The corresponding SAT problem is much larger than the original planning problem, but SAT solvers can be more efficient than theorem provers

Planning as satisfiability — declare the proper variables

- Add a time variable `Move(a,b)` → `Move(a,b,1)`
- Ground all clauses, i.e. enumerate all possible values for a given predicate's arguments
  e.g. in the monkey problem, there are 9 possible `Move` actions at time step 5:
  - `Move(a,a,5)`
  - `Move(a,b,5)`
  - ...
  - `Move(c,c,5)`
- Turn a *fully grounded* clause into a boolean variable
  `Move(a,b,12)` → `move_a_b_12`
- Express the relationships between all possible clauses as logical constraints (← this is the hard part)

Planning as satisfiability — describe the constraints

- Express the initial state and the goal state
- Describe constraints of the problem
  - no block is the table except T
  - no block is on top of itself
  - the table is never on top of anything
  - the table is always clear
  - exactly one action per time step
- Describe explanatory frame axioms
- Describe actions

Express the initial state

STRIPS:

```
Table(t), On(b,t), Clear(b), On(a, t), On(c, a), Clear(c)
```

SAT:

```
NOT clear-A-0 AND clear-B-0 AND clear-C-0
AND NOT on-A-A-0 AND NOT on-A-B-0 AND NOT on-A-C-0
  AND on-A-T-0
AND NOT on-B-A-0 AND NOT on-B-B-0 AND NOT on-B-C-0
  AND on-B-T-0
AND on-C-A-0 AND NOT on-C-B-0 AND NOT on-C-C-0
  AND NOT on-C-T-0
```

There is only one possible model for this SAT problem and it corresponds to the initial state

Express the goal state

Need to fix the number of steps in the plan ahead of time. Here say 10.

STRIPS:

```
Table(t), On(c,t), On(b,c)
```

SAT:

```
on-C-T-10 AND on-B-C-10
```

There are many possible models for this SAT problem. Each corresponding state satisfies the goal.

Constraints of the problem

- no block is the table except T

```
table-T AND NOT table-A AND NOT table-B...
```

- no block is on top of itself

```
NOT on-A-A-0 AND NOT on-B-B-0 AND ... NOT on-A-A-10 AND NOT on-
```

- the table is never on top of anything

```
NOT on-T-A-0 AND NOT on-T-B-0 AND ... NOT on-T-A-10 AND NOT on-
```

- the table is always clear

```
clear-T-0 AND clear-T-1 AND ...
```

Explanatory frame axioms

If some effect is observed, it means some action was taken

E.g. if a is on top of b at time 5 and a is not on top of b at time 4 then it means a was moved onto b at time 4.

```
(on-A-B-5 AND NOT on-A-B-4)
  => (moveFromTable-A-T-B-4 OR moveBlocks-A-C-B-4)
```

In general: a change in a state variable between time t and $t+1$ implies the disjunction of actions that change this state variable at time t.[1]

[1] It's hard to give a general definition because it depends on the underlying problem. Approximate definition on slide 19 of https://www.cs.toronto.edu/~sheila/2542/s14/material/CSC2542s14_SATPlan.pdf

Actions

Action a has preconditions p and postconditions c. a can only be taken if p is true, and if a is done then c must be true. This constraint corresponds to

$$a_t \Rightarrow (p_t \wedge c_{t+1})$$

a_t	p_t	c_{t+1}	$a_t \Rightarrow (p_t \wedge c_{t+1})$
T	T	T	T
T	T	F	F
T	F	T	F
T	F	F	F
F	T	T	T
F	T	F	T
F	F	T	T
F	F	F	T

The final model will satisfy this contraint.

- If the action was taken at time t, then the preconditions p_t must be satisfied and the postconditions c_{t+1} must hold
- If the action was not taken at time t, p_t and c_{t+1} can be true or false

Decidability

- A SAT problem is always decidable, but a general planning problem isn't
- The SAT planner assumes a fixed number of steps ahead of time
- To solve the planning problem, gradually increase the maximum number of steps until the problem becomes satisfiable

Lab session

- In the lab session, you will put translate the blocks world into a SAT problem using Z3
- You get a STRIPS representation of the blocks world, and a SATPLAN model
- Advice: when expressing constraints, describe all the grounded predicates affected by the constraint. The source of bugs is generally having too few constraints rather than too many.

Lab session

Two changes concerning how actions are represented:

`Move(X, Y, Z, T)` (moving block X from block Y to block Z at time T) becomes `Object(X, T)`, `Source(Y, T)`, `Destination(Z, T)`

This reduces the combinatorial explosion and makes some constraints easier to encode.

We also drop the distinction between `MoveBlocks`, `MoveFromTable` and `MoveToTable`. They are all represented using `Object`, `Source` and `Destination`, and the predicate `Table`.

`MoveFromTable` = move object from source to destination where object is not the table, source is the table, and destination is not the table

Outline

1. The monkey, the box and the banana
   - Planning problem
   - The STRIPS representation
   - STRIPS solver
2. Planning as satisfiability
   - Blocks world
   - Sussman anomaly
   - Kautz and Selman, 1992
   - From STRIPS to SAT
3. Partially Observable Markov Decision Process

Partially Observable Markov Decision Processes

- So far we described the environment exactly
- And the environment was deterministic: a single possible outcome for a (state, action) pair
- What if instead transitions are probabilistic: $p(s'|s,a)$ probability of moving to state s' after taking action a in state s

Partially Observable Markov Decision Processes

- Loosen the idea of a goal state, instead receive reward r at each transition $s \xrightarrow{a,r} s'$
- Instead of computing a plan, compute a policy $\pi : s \mapsto a$ that tells you which action to take in a given state
- Maximize expected reward $\sum_{t=0}^{\infty} \gamma^t \mathbb{E}[r_t]$ where the expectation is over the probabilistic transitions $p(s_{t+1}|s_t, a_t)$ when following policy π